07. RPC 서비스 분석

개요

Reth의 RPC 서비스는 외부 클라이언트가 Ethereum 노드와 상호작용할 수 있도록 하는 JSON-RPC 인터페이스를 제공합니다. 이 시스템은 모듈화된 구조로 설계되어 있으며, 다양한 전송 방식(HTTP, WebSocket, IPC)을 지원하고 여러 네임스페이스(`eth_` , `engine_` , `debug_` 등)를 통해 기능을 제공합니다.

핵심 구조체 및 트레이트

RPC 모듈 빌더

```
pub struct RpcModuleBuilder<N, Provider, Pool, Network, EvmConfig, Consensus> {
provider: Provider,
pool: Pool,
network: Network,
executor: Box<dyn TaskSpawner + 'static>,
evm_config: EvmConfig,
consensus: Consensus,
}
```

역할: RPC 모듈들을 구성하고 조립하는 메인 빌더

핵심 메서드:

- `build()` : 전송별 RPC 모듈 구성
- `build_with_auth_server()` : 인증 서버와 함께 구성
- `bootstrap_eth_api()` : 기본 Eth API 초기화

EthApi 트레이트

```
#[rpc(server, namespace = "eth")]
pub trait EthApi<TxReq, T, B, R, H> {
#[method(name = "blockNumber")]
fn block_number(&self) -> RpcResult<U256>;
#[method(name = "getBalance")]
async fn balance(&self, address: Address, block_number: Option<BlockId>) -> Rp
#[method(name = "sendTransaction")]
async fn send_transaction(&self, request: TxReq) -> RpcResult<B256>;
}
```

역할: Ethereum JSON-RPC API의 핵심 인터페이스 정의

주요 메서드:

- 블록 정보 조회: `block_by_hash()` , `block_by_number()`
- 계정 상태: `balance()` , `get_code()` , `transaction_count()`
- 트랜잭션: `send_transaction()` , `transaction_receipt()`

EngineApi 구조체

```
pub struct EngineApi<Provider, PayloadT, Pool, Validator, ChainSpec> {
```

```
inner: Arc<EngineApiInner<Provider, PayloadT, Pool, Validator, ChainSpec>>,

}
```

역할: 합의 레이어와의 통신을 위한 Engine API 구현

핵심 메서드:

- `new_payload_v1/v2/v3()` : 새로운 페이로드 처리
- `fork_choice_updated_v1/v2/v3()` : 포크 선택 업데이트
- `get_payload_v1/v2/v3()` : 페이로드 조회

주요 기능

1. 네임스페이스 관리

Reth는 다음과 같은 RPC 네임스페이스를 지원합니다:

```
pub  enum RethRpcModule {

Admin, // 노드 관리

Debug, // 디버깅 도구

Eth, // 표준 Ethereum API

Net, // 네트워크 정보

Trace, // 트랜잭션 추적

Txpool, // 트랜잭션 풀

Web3, // Web3 호환성

Engine, // 합의 레이어 통신

Reth, // Reth 전용 기능
```

```
}
```

각 네임스페이스는 독립적으로 활성화/비활성화할 수 있으며, 전송 방식별로 다른 구성이 가능합니다.

2. 전송 방식 지원

HTTP 전송

- 표준 JSON-RPC over HTTP
- CORS 지원
- 압축 지원 (gzip)
- 요청 제한 및 타임아웃

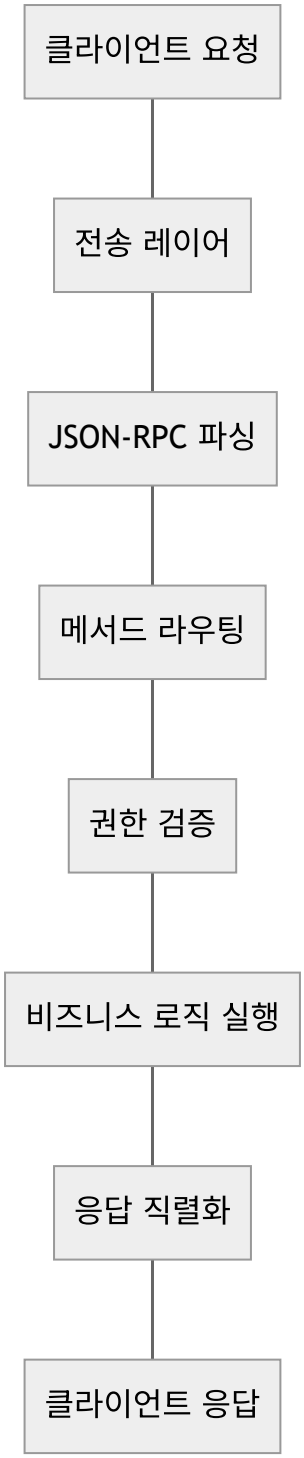
WebSocket 전송

- 실시간 이벤트 구독
- `eth_subscribe` / `eth_unsubscribe` 지원
- 연결 상태 관리

IPC 전송

- 로컬 프로세스 간 통신
- Unix 도메인 소켓 (Linux/macOS)
- Named pipe (Windows)

3. 요청 처리 파이프라인



프로세스 플로우

Eth API 요청 처리

1. **요청 수신**: 클라이언트로부터 JSON-RPC 요청 수신
2. **메서드 매칭**: `eth_getBalance` 등의 메서드명으로 핸들러 찾기
3. **매개변수 검증**: 주소, 블록 번호 등 매개변수 유효성 검사

4. **상태 조회**: `Provider` 를 통해 블록체인 상태 접근
5. **응답 생성**: 결과를 JSON-RPC 형식으로 직렬화

Engine API 처리

1. **페이로드 수신**: 합의 레이어로부터 실행 페이로드 수신
2. **검증**: 페이로드 구조 및 타임스탬프 검증
3. **실행**: `ConsensusEngineHandle` 을 통해 페이로드 실행
4. **상태 업데이트**: 실행 결과에 따른 상태 변경
5. **응답**: 실행 상태를 합의 레이어에 반환

구독 처리 (WebSocket)

```
// 새 블록 구독 예시
pub async fn subscribe_new_heads(&self) -> RpcResult<SubscriptionId> {
let subscription = self.provider
.subscribe_to_canonical_state()
.map(|notification| match notification {
CanonStateNotification::Commit { new } => {
// 새 블록 헤더를 구독자에게 전송
RpcHeader::from(new.tip().header())
}
});
Ok(self.subscription_manager.add(subscription).await?)
```

```
}
```

설정 및 커스터마이징

RPC 모듈 선택

```
// 표준 모듈 (eth, net, web3)
let  config = RpcModuleSelection::Standard;

// 모든 모듈
let  config = RpcModuleSelection::All;

// 특정 모듈 선택
let  config = RpcModuleSelection::from([
RethRpcModule::Eth,
RethRpcModule::Debug,
RethRpcModule::Trace,
]);
```

전송별 구성

```
let  transport_config = TransportRpcModuleConfig {
http: Some(RpcModuleSelection::Standard),
ws: Some(RpcModuleSelection::from([RethRpcModule::Eth])),
```

```
ipc: Some(RpcModuleSelection::All),
config: Some(RpcModuleConfig::default()),
};
```

보안 설정

```
// JWT 인증 (Engine API용)
let auth_layer = AuthLayer::new(JwtSecret::from_hex("..."))?;

// CORS 설정
let cors = CorsLayer::new()
.allow_origin("http://localhost:3000".parse::<HeaderValue>()?)
.allow_methods([Method::POST]);

// 요청 제한
let rate_limiter = RateLimiter::new(100, Duration::from_secs(60));
```

오류 처리

표준 JSON-RPC 오류

```
pub enum EngineApiError {
UnknownPayload, // -38001
```

```
InvalidForkchoiceState, // -38002

InvalidPayloadAttributes, // -38003

TooLargeRequest { len: u64 }, // -38004

UnsupportedFork, // -38005

}
```

커스텀 오류 처리

```
impl From<ProviderError> for EthApiError {

fn  from(err: ProviderError) -> Self {

match  err {

ProviderError::HeaderNotFound(_) => EthApiError::HeaderNotFound,

ProviderError::BlockNotFound(_) => EthApiError::BlockNotFound,

_ => EthApiError::Internal(err.into()),

}

}

}
```

성능 최적화

요청 배칭

```
// 여러 요청을 하나의 배치로 처리

pub  async  fn  call_many(
```

```
&self,
bundles: Vec<Bundle<TxReq>>,
state_context: Option<StateContext>,
) -> RpcResult<Vec<Vec<EthCallResponse>>> {
// 병렬 처리로 성능 향상
let futures = bundles.into_iter().map(|bundle| {
self.execute_bundle(bundle, state_context.clone())
});
try_join_all(futures).await
}
```

캐싱 전략

```
// 수수료 히스토리 캐싱
pub struct FeeHistoryCache {
cache: Arc<Mutex<LruCache<FeeHistoryKey, FeeHistory>>>,
}

impl FeeHistoryCache {
pub fn get_or_insert(&self, key: FeeHistoryKey) -> Option<FeeHistory> {
let mut cache = self.cache.lock();
cache.get(&key).cloned()
}
```

```
}
```

비동기 처리

```
// 블로킹 작업을 별도 스레드에서 처리
pub async fn trace_transaction(&self, hash: B256) -> RpcResult<TraceResults>
let _permit = self.blocking_pool_guard.acquire().await;
let provider = self.provider.clone();
let result = tokio::task::spawn_blocking(move || {
// CPU 집약적인 트레이싱 작업
provider.trace_transaction(hash)
}).await??;
Ok(result)
}
```

모니터링 및 메트릭

요청 메트릭

```
pub struct RpcRequestMetrics {
pub requests_total: Counter,
pub request_duration: Histogram,
pub active_connections: Gauge,
```

```
}

impl RpcRequestMetrics {
pub fn record_request(&self, method: &str, duration: Duration) {
self.requests_total
.with_label_values(&[method])
.inc();
self.request_duration
.with_label_values(&[method])
.observe(duration.as_secs_f64());
}
}
```

상태 모니터링

```
// RPC 서버 상태 확인
pub fn health_check(&self) -> RpcResult<HealthStatus> {
Ok(HealthStatus {
syncing: self.eth_api.syncing()?,
peer_count: self.network.peer_count(),
latest_block: self.provider.best_block_number()?,
})
```

```
}
```

관련 컴포넌트

의존성 관계

- **Provider**: 블록체인 데이터 접근
- **TransactionPool**: 트랜잭션 풀 관리
- **Network**: P2P 네트워크 정보
- **ConsensusEngine**: 합의 레이어 통신
- **PayloadBuilder**: 블록 생성

상호작용 방식

1. **데이터 조회**: Provider를 통한 블록체인 상태 접근
2. **트랜잭션 제출**: TransactionPool을 통한 트랜잭션 처리
3. **네트워크 정보**: Network를 통한 피어 및 동기화 상태
4. **실시간 업데이트**: 구독을 통한 상태 변경 알림

이러한 구조를 통해 Reth는 표준 Ethereum JSON-RPC API를 완전히 지원하면서도 확장 가능하고 성능이 우수한 RPC 서비스를 제공합니다.